

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

- ODL NO ES UN LENGUAJE DE PROGRAMACIÓN COMPLETO, ES UN **LENGUAJE DE DEFINICIÓN** INDEPENDIENTE PARA ESPECIFICAR **OBJETOS**.
- **EXTIENDE EL LENGUAJE IDL** (INTERFACE DEFINITION LANGUAGE) DESARROLLADO POR **OMG** COMO PARTE DE **CORBA** (COMMON OBJECT REQUEST BROKER ARCHITECTURE).
- **DEFINE EL OBJETO CON SUS ATRIBUTOS Y PROTOTIPOS DE MÉTODOS, NO SU IMPLEMENTACIÓN.**
- **NO ESTÁ LIGADO A LA SINTAXIS CONCRETA DE UN LENGUAJE DE PROGRAMACIÓN:**
  - **DEFINE TIPOS QUE PUEDEN IMPLEMENTARSE EN VARIOS LENGUAJES DE PROGRAMACIÓN.**
- **UN ESQUEMA DE DATOS DE OBJETO ESPECIFICADO EN ODL PUEDE SER SOPORTADO POR CUALQUIER SGBDOO QUE CUMPLA EL ESTÁNDAR ODMG.**

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

## □ NOTACIÓN:

**<definición de tipo>::=**

```
INTERFACE <nombre del tipo>(:<lista de supertipos>)
{ (<lista de propiedades del tipo>)
  (<lista de propiedades>)
  (<lista de operaciones>) }
```

**<propiedad del tipo>::=**

```
{ EXTENT <nombre de la extensión>
| KEY(S) <lista de claves> }
```

**<propiedad>::=**

```
{<especificación de atributo>
| <especificación de interrelación>
| <especificación de operación> }
```

**<especificación del atributo>::=**

```
ATTRIBUTE <tipo de dominio> <tamaño> <nombre del atributo>
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

## □ NOTACIÓN:

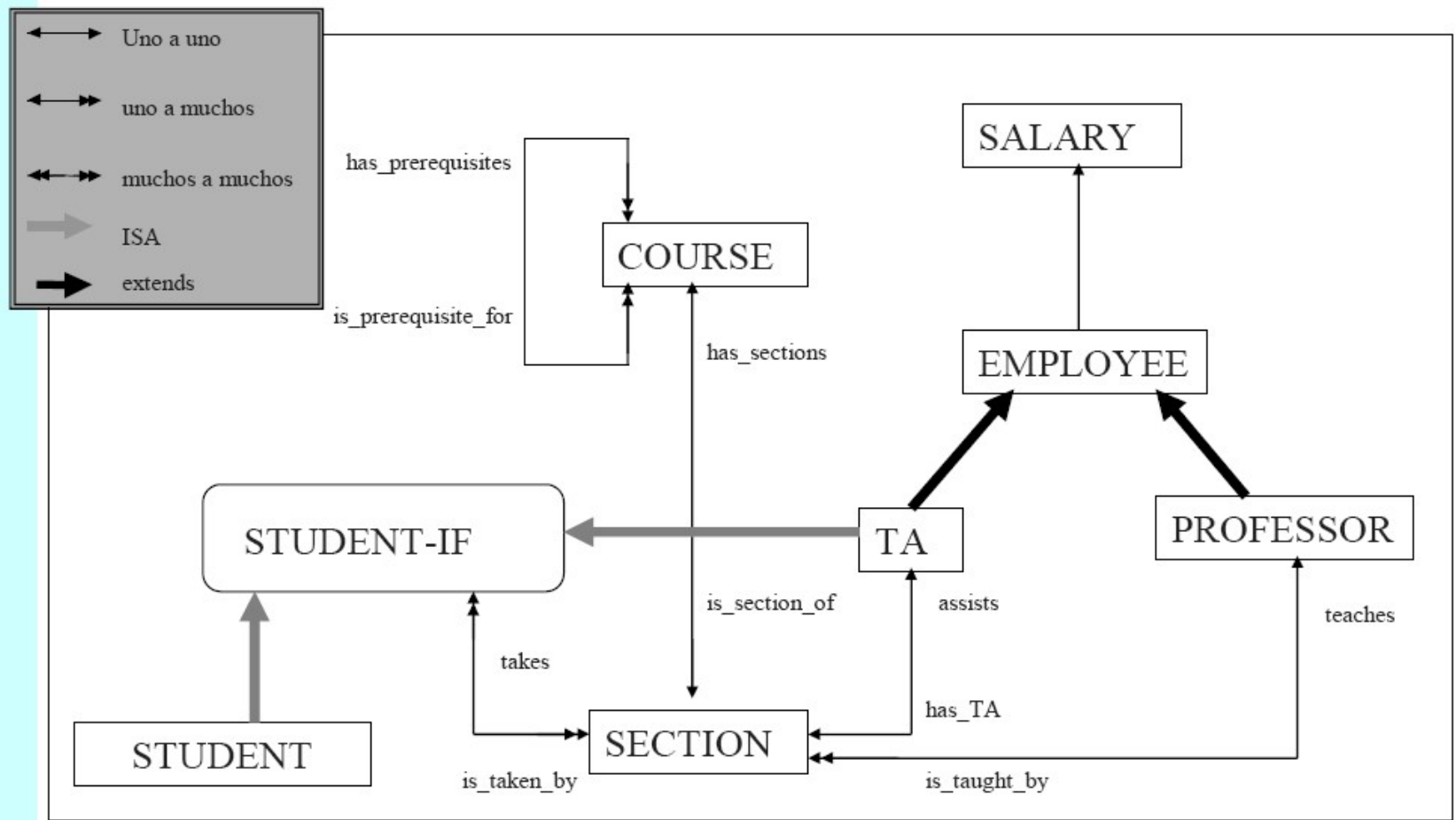
**<especificación de interrelación>::=**  
    **RELATIONSHIP** <destino del camino> <nombre del camino>  
    **INVERSE** <camino inverso>  
    [ **ORDER BY** <lista de atributos> ]

**<especificación de operación>::=**  
    <tipo devuelto> <nombre de la operación>  
    (<lista de argumentos>)  
    [ **RAISES** (<lista de excepciones>) ]

**<argumento>::=**  
    <rol> [<nombre del argumento>:] <tipo del argumento>

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

## □ EJEMPLO 1:



# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

## □ EJEMPLO 1:

```
class Course (extent courses)
{
  attribute string name;
  attribute string number;

  relationship list<Section> has_section
    inverse Section::is_section_of;
  relationship set<Course> has_prerequisites
    inverse Course::is_prerequisite_for;
  relationship set<Course> is_prerequisite_for
    inverse Course::has_prerequisites;

  boolean offer (in unsigned short semester)
    raises (alredy_offered);
  boolean drop (in unsigned short semester)
    raises (not_offered)};
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

## □ EJEMPLO 1:

```
class Section (extent sections)
{
  attribute string number;

  relationship Professor is_taught_by
    inverse Professor::teaches;
  relationship TA has_TA
    inverse TA::assists;
  relationship Course is_section_of
    inverse
      Course::has_sections;
  relationship set<Student>
    is_taken_by
      inverse Student::takes;
};
```

```
class Salary
{
  attribute float base;
  attribute float overtime;
  attribute float bonus;
};
```

```
class Employee (extent employees)
{
  attribute string name;
  attribute short id;
  attribute Salary annual_salary;

  void hire ();
  void fire()
    raises (no_such_employee)
};
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

## □ EJEMPLO 1:

```
class Professor extends Employee (extent professors)
{
  attribute enum Rank {full, associate, assistant} rank;

  relationship set<Section> teaches
    inverse Section::is_taught_by;

  short grant_tenure()
    raises (ineligible_for_tenure)};
```



# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

## □ EJEMPLO 1:

```
interface Student-IF
{struct Address {string college, string room_number}
attribute string name;
attribute string student_id;
attribute Address dorm_address;

relationship set<Section> takes
    inverse Section::is_taken_by;

boolean register_for_course (in unsigned short course,
                             in unsigned short Section)
    raises (unsatisfied_prerequisites, section_full, course_full);
boolean drop_course (in unsigned short course)
    raises (not_registered_for_that_course)
void assign_major (in unsigned short Departament);
short transfer (in unsigned short old_section,
               in unsigned short new_section)
    raises (section_full, not_registered_in_section)};
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

## □ EJEMPLO 1:

```
class TA extends Employee:Student-IF
    (extent Employee)
{
    relationship Section assists
        inverse Section::has_TA;

    attribute string name;
    attribute string student_id;
    attribute Address dorm_address;

    relationship set<Section> takes
        inverse Section::is_taken_by;
};
```

```
class Student:Student-IF
    (extent Students)
{
    attribute string name;
    attribute string student_id;
    attribute Address dorm_address;

    relationship set<Section> takes
        inverse Section::is_taken_by;
};
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

## □ EJEMPLO 2:

```
class Persona
    (extent personas key dni)
{
    /* Definición de atributos */
    attribute struct Nom_Persona {string nombre_pila, string apellido1,
                                   string apellido2} nombre;
    attribute string dni;
    attribute date fecha_nacim;
    attribute enum Genero{F,M} sexo;
    attribute struct Direccion {string calle, string cp, string ciudad}
                                   direccion;
    /* Definición de operaciones */
    float edad();
}
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

```
class Profesor extends Persona
    (extent profesores)
{
    /* Definición de atributos */
    attribute string categoria;
    attribute float salario;
    attribute string despacho;
    atributo string telefono;
    /* Definición de relaciones */
    relationship Departamento trabaja_en
        inverse Departamento::tiene_profesores;
    relationship Set<EstudianteGrad> tutoriza
        inverse EstudianteGrad::tutor;
    relationship Set<EstudianteGrad> en_comite
        inverse EstudianteGrad::comite;
    /* Definición de operaciones */
    void aumentar_salario(in float aumento);
    void promocionar(in string nueva_categoria);
}
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

```
class Estudiante extends Persona
    (extent estudiantes)
{
/* Definición de atributos */
    attribute string titulacion;
/* Definición de relaciones */
    relationship set<Calificacion> ediciones_cursadas
        inverse Calificacion::estudiante;
    relationship set<EdicionActual> matriculado
        inverse EdicionActual::estudiantes_matriculados;
/* Definición de operaciones */
    float nota_media();
    void matricularse(in short num_edic) raises(edicion_no_valida, edicion_llena);
    void calificar(in short num_edic; in float nota)
        raises(edicion_no_valida, nota_no_valida);
};
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

```
class Calificacion
    (extent calificaciones)
{
/* Definición de atributos */
    attribute float nota;
/* Definición de relaciones */
    relationship Edicion edicion inverse Edicion::estudiantes;
    relationship Estudiante estudiante
        inverse Estudiante::ediciones_cursadas;
};
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

```
class EstudianteGrad extends Estudiante
    (extent estudiantes_graduados)
{
/* Definición de atributos */
    attribute set<Titulo> titulos;
/* Definición de relaciones */
    relationship Profesor tutor inverse Profesor::tutoriza;
    relationship set<Profesor> comite inverse Profesor::en_comite;
/* Definición de operaciones */
    void asignar_tutor(in string apellido1; in string apellido2)
        raises(profesor_no_valido);
    void asignar_miembro_comite(in string apellido1; in string apellido2)
        raises(profesor_no_valido);
};
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

```
class Titulo
{
/* Definición de atributos */
    attribute string escuela;
    attribute string titulo;
    attribute string año;
};
```



# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

```
class Departamento
    (extent departamentos key nombre)
{
    /* Definición de atributos */
    attribute string nombre;
    attribute string telefono;
    attribute string despacho;
    attribute string escuela;
    attribute Profesor director;
    /* Definición de relaciones */
    relationship set<Profesor> tiene_profesores
        inverse Profesor::trabaja_en;
    relationship set<Curso> oferta
        inverse Curso::ofertado_por;
};
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

```
class Curso
    (extent cursos key num_curso)
{
/* Definición de atributos */
    attribute string nombre;
    attribute string num_curso;
    attribute string descripcion;
/* Definición de relaciones */
    relationship set<Edicion> tiene_ediciones
        inverse Edicion::de_curso;
    relationship Departamento ofertado_por inverse Departamento::oferta;
};
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

```
class Edicion
    (extent ediciones)
{
/* Definición de atributos */
    attribute short num_edic
    attribute string año;

    attribute enum Semestre{Primero,Segundo} semestre;
/* Definición de relaciones */
    relationship set<Calificacion> estudiantes
        inverse Calificacion::edicion;
    relationship Curso de_curso inverse Curso::tiene_ediciones;
};
```

# LENGUAJE DE DEFINICIÓN DE OBJETOS ODL

```
class EdicionActual extends Edicion
    (extent ediciones_actuales)
{
    /* Definición de relaciones */
    relationship set<Estudiante> estudiantes_matriculados
        inverse Estudiante::matriculado;
    /* Definición de operaciones */
    void matricular_estudiante(in string dni)
        raises(estudiante_no_valido, edicion_llena);
};
```